

Мики Тебека

ХОРОШО ЛИ ВЫ ЗНАЕТЕ



Мики Тебека

Хорошо ли вы знаете Go?

Go Brain Teasers

Exercise Your Mind

Miki Tebeka
edited by Margaret Eldridge



Хорошо ли вы знаете Go?

25 задач для разминки ума

Мики Тебека
под редакцией Маргарет Элдридж



Москва, 2025

УДК 004.438Go
ББК 32.973.2
Т29

Мики Тебека

Т29 Хорошо ли вы знаете Go? / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2025. – 112 с.: ил.

ISBN 978-5-93700-362-1

Язык программирования Go прост, но имеет свои странные особенности. Исследуя их на примере специально подобранных задач, вы перейдете на следующий уровень осмысленного использования языка. Данная книга состоит из 25 небольших, но нетривиальных программ, которые сопровождаются детальными пояснениями. Но не спешите открывать страницу с ответом! Попробуйте выполнить программу и самостоятельно объяснить полученный результат. Предполагается, что вы в какой-то мере знакомы с Go и имеете опыт программирования на этом языке.

Издание адресовано как опытным программистам, желающим отточить свои навыки, так и новичкам, стремящимся к профессиональному росту.

УДК 004.438Go
ББК 32.973.2

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN (анг.) 978-1-68050-899-4
ISBN (рус.) 978-5-93700-362-1

Copyright © 2021 The Pragmatic Programmers, LLC.
© Оформление, издание, перевод, ДМК Пресс, 2025

*Посвящается Шэрон,
которая терпела меня во время карантина
и двадцать лет до того*

Оглавление

Предисловие от издательства	9
Благодарности.....	10
Предисловие.....	11
Об авторе	13
О коде	14
О вас	15
И еще одно.....	16
Задача 1. Число π	17
Задача 2. С пустыми руками	21
Задача 3. Однажды в Кракове.....	23
Задача 4. Нет у меня ничего.....	27
Задача 5. Сыроядение.....	29
Задача 6. Мы уже приехали?	31
Задача 7. Могут ли числа лгать?	33
Задача 8. Сортировка во сне.....	37
Задача 9. Точно вовремя	41
Задача 10. Простое дописывание	45

Задача 11. Что в журнале?	49
Задача 12. Странное число	53
Задача 13. Целые числа и диапазоны	57
Задача 14. Несколько личин	63
Задача 15. Повесть о двух городах	65
Задача 16. Что у нас в канале?	69
Задача 17. **Int** ересная строка	73
Задача 18. Отложенная работа	77
Задача 19. Ошибка или не ошибка	81
Задача 20. Что в строке?	83
Задача 21. Двойняшки	87
Задача 22. Отсчитайте мне миллион	91
Задача 23. Кто следующий?	97
Задача 24. Ох уж эти флаги	99
Задача 25. Даю разрешение	103
Предметный указатель	107

Предисловие от издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в основном тексте или программном коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу dmkpress@gmail.com, и мы исправим это в следующих тиражах.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Благодарности

Я благодарен за все отзывы: от указаний на ошибки в коде до исправления грамматических ошибок.

Ниже приводится список людей, помогавших мне; приношу извинения всем, кого я забыл упомянуть.

- Ади Тебека – за корректуру и комментарии.
- Дэн Аллен – за помощь на форумах `asciidoc`. Я пользовался этим замечательным сайтом при написании первого варианта книги.
- Дэйв Чини – за вступительное слово.
- Дэвид Бордейник – за комментарии.
- Игон Элбр – за нарисованных им прекрасных сусликов, которых он поместил также в лицензию CC0.
- Илад Эйял – за корректуру.
- Элиран Бивас – за корректуру и комментарии.
- Джаред Дэвис – за отзыв.
- Рэн Тэвори – за комментарии.
- Йони Дэвидсон – за комментарии.

Я также благодарен Брэду Фицпатрику и другим, кто публикует тесты по Go, откуда я черпал идеи для этих задач.

Предисловие

Язык программирования Go прост, но, как и у любого другого языка, у него есть свои причуды. В этой книге причуды используются как дополнительная возможность чему-то научиться. Осознав пробелы в своих знаниях, вы станете лучше выполнять свою работу.

Есть немало исследований, доказывающих, что люди, делающие ошибки во время обучения, в итоге обучаются лучше, чем те, кто ошибок не делает. Если вы будете иметь это в виду при исправлении дефектов в программе, то охота за ошибками будет доставлять вам удовольствие, а после исправления каждой вы, как разработчик, будете становиться лучше.

Эти задачи помогут вам избежать ошибок. Некоторые задачи взяты из моего личного опыта поставки ПО с ошибками, а некоторые – из опыта других людей.

Все мы, профессионалы, любим решать задачки. Но вы можете также использовать их, чтобы произвести впечатление на коллег, устроить интеллектуальный бой и вместе стать лучше.

В основе многих задач лежат вопросы, которые я предлагал на конференциях и профессиональных собраниях. Оказалось, что людям это нравится и оживляет атмосферу.

В начале каждой главы приводится короткая программа на Go, и вам предлагается угадать, что произойдет при попытке ее выполнить. Ответы могут быть такими:

- не компилируется;
- паникует;
- взаимоблокировка;
- что-то печатается (например, `[1 2 3]`).

Версия Go



При выполнении кода я использовал версию Go 1.14.1, но в будущих версиях результат *может* быть иным.

Прежде чем смотреть ответ и объяснения, попытайтесь угадать результат самостоятельно. А потом я рекомендую выполнить код и посмотреть, что получится, и лишь после этого переходить к чтению решения и объяснения. Я преподаю программирование много лет и нахожу такой порядок действий весьма эффективным.

Об авторе

Мики Тебека получил степень бакалавра компьютерных наук в университете имени Бен-Гуриона. Там же он прослушал курс вычислительной лингвистики.

У Мики страсть к преподаванию и наставничеству. Он ведет много семинаров по различным техническим предметам по всему миру и был наставником многих начинающих разработчиков на их пути к успеху. Мики создал ряд собственных проектов с открытым исходным кодом и принимает участие еще в нескольких, в том числе Python. Он пишет на Python более 23 лет.

Мики является автором книг «Pandas Brain Teasers», «Python Brain Teasers» и «Forging Python», он автор, зарегистрированный на сайте LinkedIn Learning, и организатор мероприятий Go Israel Meetup, GopherCon Israel и PyData Israel Conference.

О коде

Код, приведенный в книге, можно найти по адресу
<https://pragprog.com/titles/d-gobrain/go-brain teasers/>.

Я старался максимально сократить код и удалял все не имеющее прямого отношения к задаче. Обычно так код *не* пишется.

О вас

Предполагается, что вы в какой-то мере знакомы с Go и имеете опыт программирования на этом языке. Эта книга не учебник по Go. Если вы совсем не знаете Go, боюсь, что эти задачи не для вас.

И еще одно

Когда будете прорабатывать задачи, представляйте себя Нэнси Дрю, Шерлоком Холмсом или еще каким-нибудь детективом, который пытается раскрыть убийство, а убийцей являетесь *вы*. Примите такую точку зрения:

«Отладка программы сродни работе кинематографического детектива в фильме, расследующего убийство, совершенное вами же» (Филипе Фортес).

Мне кажется, что при таком отношении проще понять, что происходит, да к тому же и веселее. Так что получайте удовольствие от разгадывания головоломок – глядишь, освоите пару-другую новых трюков.

Если захотите узнать больше, пишите на адрес info@353solutions.com, и мы организуем практический семинар на интересующую вас тему. Также на сайте <https://www.353solutions.com> есть богатая подборка семинаров.

Оставайтесь любознательными и докапывайтесь до сути вещей!

Мики Тебека, март 2020

Задача 1

Число π

pi.go

```
package main

import (
    "fmt"
)

func main() {
    var n = 355 / 113.0
    fmt.Println(n)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `3.1415929203539825`

Обсуждение

Тут у нас две достойные удивления вещи: во-первых, оказывается, что `π` – допустимый идентификатор, а во-вторых – что выражение `355 / 113.0` компилируется.

Начнем с `π`. В спецификации языка Go по поводу идентификаторов написано:

«Идентификаторы именуют такие программные сущности, как переменные и типы. Идентификатор представляет собой последовательность одной или более букв и цифр. Первый символ идентификатора должен быть буквой».

Буквами могут быть любые буквы Юникода, в том числе `π`. Это, может быть, и забавно, но определенно усложнит жизнь вашим коллегам. Я легко могу набрать `π` в редакторе Vim, в котором работаю, но в большинстве редакторов и IDE потребуется больше усилий.

Единственное место, где, на мой взгляд, идентификаторы с символами Юникода могут быть полезны, – запись математических формул в коде. Во всех остальных случаях ограничьтесь старым добрым ASCII.

Теперь что касается `355 / 113.0`. Система типов Go не допускает деления (или любых других математических операций) между целым (`355`) и числом с плавающей точкой (`113.0`). Но здесь в правой части оператора присваивания `=` находятся константы, а не переменные. Тип константы определяется в момент использования; в данном случае компилятор преобразует `355` в число с плавающей точкой, чтобы выполнить операцию.

Если вы сначала присвоите значения `355` и `113.0` переменным и попыдаете выполнить ту же программу, то ничего не получится. Следующий код не откомпилируется:

```
pi_var.go
```

```
package main

import (
    "fmt"
)

func main() {
    a, b := 355, 113.0
    var p = a / b
    fmt.Println(p)
}
```

Для дополнительного чтения

Спецификация констант

golang.org/ref/spec#Constants.

Константы

blog.golang.org/constants.

Ardan Labs «Introduction to Numeric Constants in Go»

ardanlabs.com/blog/2014/04/introduction-to-numeric-constants-in-go.html.

«Википедия» о приближениях π

en.wikipedia.org/wiki/Approximations_of_%CF%80.

Идентификаторы в спецификации языка Go

golang.org/ref/spec#Identifiers.

Задача 2

С пустыми руками

empty_map.go

```
package main

import (
    "fmt"
)

func main() {
    var m map[string]int
    fmt.Println(m["errors"])
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 0

Обсуждение

Неинициализированное отображение принимает значение `nil`. Некоторые операции над отображениями в Go безопасны относительно `nil`, т. е. работают не паникуя.

И метод `len`, и доступ к значению (например, `m["errors"]`) работают, даже когда отображение равно `nil`. В этом случае `len` возвращает 0, а доступ по ключу определен следующим образом:

- если ключ принадлежит отображению, то возвращается ассоциированное с ним значение;
- если ключ не принадлежит отображению, то возвращается нулевое значение типа значения.

В данном случае ключ `"errors"` отсутствует в отображении, и мы получаем нулевое значение типа `int`, т. е. 0.

Такое поведение удобно. Если мы хотим подсчитать элементы (например, частоту слов в документе), то можем писать `m[word]++`, не думая о том, присутствует `word` в отображении `m` или нет.

Но тогда возникает вопрос. Как узнать, является ли полученный нуль значением существующего в отображении ключа, или это просто нулевое значение типа значения? Для этого нужно воспользоваться идиомой `запятая, ok`.

Если написать `val, ok := m[key]`, то `ok` будет равно `true`, если ключ `key` присутствует в отображении, и `false`, если его там нет.

В Go есть и другие безопасные относительно `nil` типы. Например, мы можем запросить длину нулевого среза, получить значение из пустого канала (хотя при этом программа будет навсегда заблокирована) и т. д.

Для дополнительного чтения

Отображения Go в действии

blog.golang.org/maps.

Об использовании отображений на сайте Effective Go

golang.org/doc/effective_go.html#maps.

Нулевое значение в спецификации Go

golang.org/ref/spec#The_zero_value.

Задача 3

Однажды в Кракове

city.go

```
package main

import (
    "fmt"
)

func main() {
    city := "Kraków"
    fmt.Println(len(city))
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 7

Обсуждение

В слове `Kraków` вроде бы шесть символов. Так почему же печатается 7? Всему причиной... история.

Первые компьютеры были сконструированы в англоязычных странах: Великобритании и США. Столкнувшись с необходимостью кодировать текст для компьютеров, понимающих только биты, разработчики придумали следующую схему. Будем использовать для представления символа байт (8 бит). Например, букве `a` соответствует 97 (`01100001`), букве `b` – 98 и т. д. Для английского алфавита, содержащего 26 строчных букв, 26 заглавных букв и 10 цифр, одного байта достаточно. Остается даже место для некоторых специальных символов (например, 9 для знака табуляции). Такая схема называется *кодировкой ASCII*.

Шло время, компьютеры начали использовать и в других странах, которые захотели применять свой язык. Код ASCII для этой цели не подходил: одного байта недостаточно для представления всех букв разных языков. Это стало причиной появления других схем кодирования, самой известной из которых является UTF-8. Роб Пайк, один из проектировщиков Go, также принимал участие в проектировании UTF-8.

В Go строки представлены в кодировке UTF-8. Это означает, что литера (называемая *руной*) может иметь длину от 1 до 4 байт. Запросив длину строки в Go, вы получите размер в байтах. В данном случае руна `ó` занимает 2 байта, потому-то общая длина строки равна 7.

Если вы хотите знать, сколько рун в строке, то должны воспользоваться пакетом `unicode/utf8`.

```
cityu.go
```

```
package main
```

```
import (  
    "fmt"  
    "unicode/utf8"  
)
```

```
func main() {  
    city := "Kraków"  
    fmt.Println(utf8.RuneCountInString(city))  
}
```

Для дополнительного чтения

Строки, байты, руны и литеры в Go
blog.golang.org/strings.

Юникод и вы
betterexplained.com/articles/unicode/.

«Википедия» о Юникоде
en.wikipedia.org/wiki/Unicode.

«Википедия» об ASCII
en.wikipedia.org/wiki/ASCII.

«Википедия» о UTF-8
en.wikipedia.org/wiki/UTF-8.

Задача 4

Нет у меня ничего

nil.go

```
package main

import (
    "fmt"
)

func main() {
    n := nil
    fmt.Println(n)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Этот код не компилируется.

Обсуждение

`nil` не тип, а зарезервированное слово. Переменная, инициализированная значением `nil`, должна иметь тип. Например, если вы измените присваивание, написав `var n *int = nil`, то код откомпилируется, потому что теперь `n` имеет тип.

Ниже описано несколько мест, где используется `nil`:

- нулевое значение типов `map`, среза и `chan` равно `nil`;
- сравнивать срезы и отображения с помощью оператора `==` нельзя; их можно сравнивать только с `nil`;
- отправка данных в канал `nil` или их получение из канала `nil` ведет к вечной блокировке. Этим можно воспользоваться, чтобы избежать активного ожидания.

Для дополнительного чтения

«*Understanding nil*» из книги Dave Cheney «*Practical Go*»

dave.cheney.net/practical-go/presentations/gophercon-israel.html#nil.

Почему в Go разрешены `nil`-каналы?

medium.com/justforfunc/why-are-there-nil-channels-in-go-9877cc0b2308.

Dave Cheney «*Channel Axioms*»

dave.cheney.net/2014/03/19/channel-axioms.

Задача 5

Сыроядение

raw.go

```
package main

import (
    "fmt"
)

func main() {
    s := `a|tb`
    fmt.Println(s)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `a\tb`

Обсуждение

В спецификации Go сказано¹:

«Существует две формы: исходные (“сырые”) строковые литералы и интерпретированные строковые литералы».

Исходные строки заключаются в обратные апострофы, а интерпретированные – в кавычки. В исходных строках знак `\` не имеет специального значения, поэтому `'\t'` – две литеры: обратная косая черта и буква `t`. Если бы это была интерпретированная строка, то `\t` интерпретировалась бы как знак табуляции.

Помимо обычных знаков `\t` (табуляция), `\n` (переход на новую строку) и т. п., в интерпретированных строках можно использовать кодовые точки Юникода. Так, `fmt.Println("\u2122")` напечатает TM.

Один из наиболее распространенных случаев использования исходных строк – создание многострочных строк:

```
// HTTP-запрос
request := `GET / HTTP/1.1
Host: www.353solutions.com
Connection: Close`
```

Для дополнительного чтения

Литералы в спецификации Go

golang.org/ref/spec#String_literals.

Строки, байты, руны и литеры в Go

blog.golang.org/strings.

Спецификация строк

golang.org/ref/spec#String_literals.

«Википедия» об управляющих символах ASCII

en.wikipedia.org/wiki/Control_character#In_ASCII.

¹ https://golang.org/ref/spec#String_literals.

Задача 6

Мы уже приехали?

time.go

```
package main

import (
    "fmt"
    "time"
)

func main() {
    timeout := 3
    fmt.Printf("before ")
    time.Sleep(timeout * time.Millisecond)
    fmt.Println("after")
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа не компилируется.

Обсуждение

Видя предложение `timeout := 3`, компилятор Go производит вывод типа. В данном случае он приходит к выводу, что `timeout` имеет тип `int`. Затем мы умножаем `timeout` (типа `int`) на `time.Millisecond` (типа `time.Duration`), что запрещено.

Исправить ошибку можно несколькими способами.

- Изменить тип `timeout` на `time.Duration`:

```
var timeout time.Duration = 3
```

- Сделать `timeout` константой, тогда ее тип будет выведен в контексте использования:

```
const timeout = 3
```

- Преобразовать тип `timeout` в `time.Duration`:

```
time.Sleep(time.Duration(timeout) * time.Millisecond)
```

Для дополнительного чтения

«Википедия» о выводе типов

en.wikipedia.org/wiki/Type_inference.

Вывод типов на сайте Go Tour

tour.golang.org/basics/14.

`go/types`: средство проверки типов в Go на сайте `golang/example`

<https://github.com/golang/example/tree/master/gotypes>.

`time.Duration`

golang.org/pkg/time/#Duration.

Преобразования типов в спецификации Go

golang.org/ref/spec#Conversions.

Задача 7

Могут ли числа лгать?

float.go

```
package main

import (
    "fmt"
)

func main() {
    n := 1.1
    fmt.Println(n * n)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `1.21000000000000002`

ОБСУЖДЕНИЕ

Вы, вероятно, ожидали увидеть `1.21`, и это математически правильный ответ.

Некоторые начинающие разработчики, видя этот или подобный результат, пишут в форумах: «Мы нашли ошибку в Go!» Обычно на это отвечают: «Читай руководство» (RTFM).

«Арифметика с плавающей точкой – как квантовая физика: чем пристальнее всматриваешься, тем запутанней становится» (Грант Эдвардс).

Основная идея заключается в том, что, имея дело с числами с плавающей точкой, мы жертвуем точностью ради скорости (т. е. обманываем). Ой, ну не падайте в обморок: такие компромиссы встречаются в информатике сплошь и рядом.

Результат, который вы видите, находится в полном соответствии со спецификацией операций с плавающей точкой. Выполнив аналогичный код в Python, Java, C и т. д., вы увидите то же самое.

Если вы хотите разобраться, как работает арифметика с плавающей точкой, обратитесь к ссылкам ниже. Но главное – запомните, что они не точны, и точность тем хуже, чем числа больше.

Отсюда, в частности, следует, что при сравнении чисел с плавающей точкой нужно проверять на «приближенное равенство» и выбирать приемлемую в данной ситуации погрешность. В библиотеках тестирования, например `stretchr/testify`, имеются готовые функции (скажем, `InDelta`) для этой цели.

У операций с плавающей точкой есть и другие странности. Например, имеется специальное значение `NaN` (*нечисло*). `NaN` не равно никакому числу, *в том числе самому себе*. Следующий код напечатает `false`:

```
fmt.Println(math.NaN() == math.NaN())
```

Чтобы узнать, получили ли вы `NaN`, следует использовать специальную функцию `math.IsNaN`.

Если вам необходима большая точность, обратитесь к пакету `math/big` или таким внешним пакетам, как `shopspring/decimal`.

Для дополнительного чтения

Julia Evans «Floating point zine»

twitter.com/b0rk/status/986424989648936960.

Что каждый специалист по информатике должен знать об арифметике с плавающей точкой

docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html.

«Википедия» о спецификации операций с плавающей точкой

en.wikipedia.org/wiki/IEEE_754.

Задача 8

Сортировка во сне

sleep_sort.go

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup
    for _, n := range []int{3, 1, 2} {
        wg.Add(1)
        go func() {
            defer wg.Done()
            time.Sleep(time.Duration(n) * time.Millisecond)
            fmt.Printf("%d ", n)
        }()
    }
    wg.Wait()
    fmt.Println()
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 2 2 2

ОБСУЖДЕНИЕ

Вы, надо думать, ожидали получить 1 2 3. Каждая go-подпрограмма¹ спит `n` мс, а затем печатает `n`. Функция `wg.Wait()` ждет завершения всех go-подпрограмм, после чего программа печатает знак перехода на новую строку.

Однако каждая go-подпрограмма использует одно и то же `n`, определенное в строке 11. Это называется *замыканием*.

Именно по этой причине go-подпрограммы (и отложенные функции) вызываются с параметрами.

Решить эту проблему можно двумя способами. Первый, на мой взгляд предпочтительный, – передать `n` в качестве параметра go-подпрограмме.

sleep_sort_param.go

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup
    for _, n := range []int{3, 1, 2} {
        wg.Add(1)
        go func(i int) {
            defer wg.Done()
            time.Sleep(time.Duration(n) * time.Millisecond)
            fmt.Printf("%d ", i)
        }(n)
    }
    wg.Wait()
    fmt.Println()
}
```

¹ Часто встречается «перевод» *горутина*, но меня от этого слова тошнит. Ведь не говорим же мы ни «сабрутина», ни «корутина». – Прим. перев.

Во втором решении используется тот факт, что в Go каждое употребление `{` открывает новую область видимости переменных.

sleep_sort_scope.go

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup
    for _, n := range []int{3, 1, 2} {
        n := n // <1>
        wg.Add(1)
        go func() {
            defer wg.Done()
            time.Sleep(time.Duration(n) * time.Millisecond)
            fmt.Printf("%d ", n)
        }()
    }
    wg.Wait()
    fmt.Println()
}
```

Теперь `n` – новая переменная, которая существует только в области видимости цикла `for` и, что для нас важнее, в замыкании го-подпрограммы. Она «затеняет» внешнюю переменную `n`, определенную в строке 11.

Для дополнительного чтения

Замыкания функций на сайте Go Tour
tour.golang.org/moretypes/25.

«Википедия» о замыканиях
[en.wikipedia.org/wiki/Closure_\(computer_programming\)](http://en.wikipedia.org/wiki/Closure_(computer_programming)).

Задача 9

Точно вовремя

time_eq.go

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "time"
)

func main() {
    t1 := time.Now()
    data, err := json.Marshal(t1)
    if err != nil {
        log.Fatal(err)
    }

    var t2 time.Time
    if err := json.Unmarshal(data, &t2); err != nil {
        log.Fatal(err)
    }
    fmt.Println(t1 == t2)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `false`

ОБСУЖДЕНИЕ

Возможно, вы ожидали, что программа завершится аварийно, потому что в формате JSON нет типа `time`. Или думали, что сравнение будет успешным.

В Go пакет `encoding/json` позволяет определить собственную JSON-сериализацию для типов, не поддерживаемых JSON. Для этого необходимо реализовать интерфейсы `json.Marshaler` и `json.Unmarshaler`. Тип `Go time.Time` реализует эти интерфейсы, сериализуя себя в строки согласно документу RFC 3339.

Набор типов в JSON очень ограничен. Например, в нем есть только числа с плавающей точкой, что может приводить к неожиданным результатам, если интерфейс десериализации пустой.

`json_float.go`

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
)

func main() {
    n1 := 1
    data, err := json.Marshal(n1)
    if err != nil {
        log.Fatal(err)
    }

    var n2 interface{}
    if err := json.Unmarshal(data, &n2); err != nil {
        log.Fatal(err)
    }

    fmt.Printf("n1 is %T, n2 is %T\n", n1, n2)
}
```

Эта программа напечатает `n1 is int, n2 is float64`.

После выполнения JSON-сериализации вы вправе ожидать, что моменты времени будут равны. В документации по пакету `time` сказано следующее (курсив мой):

«Операционные системы предлагают как “системные часы”, которые могут изменяться в результате синхронизации времени, так и “монотонные часы”, которые при такой синхронизации не изменяются. Общее правило заключается в том, что системные часы показывают время, а монотонные часы измеряют время. В этом пакете API не разбивается на части, а вместо этого `time.Now` содержит как показание системных часов, так и показание монотонных часов...»

Монотонные часы служат для измерения промежутков времени. Они призваны избежать проблем, которые могли бы возникнуть, если во время измерения компьютер переходит на летнее время. Само по себе показание монотонных часов не значит ничего, полезна только разность двух показаний.

Когда для сравнения значений типа `time.Time` используется оператор `==`, Go сравнивает поля структуры `time.Time`, включая показания монотонных часов. Однако когда Go сериализует `time.Time` в формат JSON, показание монотонных часов не учитывается. При обратном считывании времени в переменную `t2` показаний монотонных часов не будет, и сравнение не пройдет.

Решение проблемы описано в документации по типу `time.Time`:

«В общем случае отдавайте предпочтение `t.Equal(u)` перед `t == u`, потому что в `t.Equal` используется более точное сравнение и корректно обрабатывается случай, когда только в одном аргументе имеется показание монотонных часов».

Для дополнительного чтения

Формат JSON

json.org.

Ложь, в которую верят программисты, когда речь идет о времени

infiniteundo.com/post/25326999628/falsehoods-programmers-believe-about-time.

Документация по пакету Time

golang.org/pkg/time/#Time.

Документ RFC 3339

ietf.org/rfc/rfc3339.txt.

«Википедия» о стандарте ISO 8601

en.wikipedia.org/wiki/ISO_8601.

Задача 10

Простое дописывание

append.go

```
package main

import (
    "fmt"
)

func main() {
    a := []int{1, 2, 3}
    b := append(a[:1], 10)
    fmt.Printf("a=%v, b=%v\n", a, b)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `a=[1 10 3]`, `b=[1 10]`

ОБСУЖДЕНИЕ

Чтобы понять, почему результат именно таков, нам придется разобраться в том, как реализованы срезы и как работает `append`.

Заглянув в файл `src/runtime/slice.go` в исходном коде Go, мы увидим

```
type slice struct {
    array unsafe.Pointer
    len int
    cap int
}
```

Здесь `slice` – структура с тремя полями, `array` – указатель на внутренний массив, в котором хранятся данные, `len` – длина среза, а `cap` – емкость массива `array`.

На рисунке ниже показана иллюстрация.



Срез (например, `s1[3:7]`) считается *полуоткрытым*, т. е. первый индекс включается, а второй – нет. В срез `s2` входят индексы 3, 4, 5 и 6, т. е. его длина (`len`) равна 4.

Емкость – это количество элементов от начала среза до конца внутреннего массива. В случае `s2` мы начинаем отсчет с 3 и можем дойти до 10, т. е. емкость равна 7.

Узнать длину и емкость позволяют встроенные функции `len` и `cap`: `len(s2) == 4`, а `cap(s2) == 7`.

Далее поговорим о том, как работает `append`. При вызове `append` проверяется емкость. Если места достаточно, то `append` добавит элемент в конец внутреннего массива и вернет срез, указывающий на тот же

самый массив, но большей длины. Если места не осталось, то `append` создает новый больший массив, копирует в него старый, добавляет элемент в новый массив и возвращает указывающий на него срез.

Вот одна из возможных реализаций `append`:

```
func Append(items []int, i int) []int {
    if len(items) == cap(items) { // Во внутреннем массиве нет места
        // В Go реализована лучшая эвристика, чем увеличение
        // на 1 при каждом append
        newItems := make([]int, len(items)+1)
        copy(newItems, items)
        items = newItems
    } else {
        items = items[:len(items)+1]
    }
    items[len(items)-1] = i
    return items
}
```

Вот теперь поведение программы понятно. Нас интересует строка `b := append(a[:1], 10)`. `a[:1]` создает срез длины 1 и емкости 3. `append` видит, что места достаточно, и изменяет внутренний массив, помещая `10` в позицию с индексом 1.

`a` и `b` указывают на один и тот же внутренний массив. Длина `a` равна 3, а длина `b` равна 2. Поэтому и выводится `a=[1 10 3]`, `b=[1 10]`.

Для дополнительного чтения

Внутреннее устройство срезов в блоге Go
blog.golang.org/slices.

О срезах на сайте Effective Go
golang.org/doc/effective_go.html#slices.

Приемы работы со срезами на сайте Go Wiki
github.com/golang/go/wiki/SliceTricks.

Задача 11

Что в журнале?

struct.go

```
package main

import (
    "fmt"
    "time"
)

// Log описывает сообщение в журнале
type Log struct {
    Message string
    time.Time
}

func main() {
    ts := time.Date(2009, 11, 10, 0, 0, 0, 0, time.UTC)
    log := Log{"Hello", ts}
    fmt.Printf("%v\n", log)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 2009-11-10 00:00:00 +0000 UTC

ОБСУЖДЕНИЕ

Спецификатор `%v` печатает все поля структуры:

```
struct_pt.go
```

```
package main

import (
    "fmt"
)

// Point – точка на плоскости
type Point struct {
    X int
    Y int
}

func main() {
    p := Point{1, 2}
    fmt.Printf("%v\n", p) // {1 2}
}
```

На первый взгляд, должно быть напечатано `{Hello 2009-11-10 00:00:00 +0000 UTC}`. Но печатается другое – из-за того, как определена структура `Log`. Второе поле в ней не имеет имени, только тип. Это называется *встраиванием* и означает, что тип `Log` включает все поля и методы, имеющиеся в типе `time.Time`.

В типе `time.Time` определен метод `String()`, т. е. он реализует интерфейс `fmt.Stringer`. А поскольку этот тип встроен в `Log`, последний также обладает методом `String()`. Если параметр, переданный функции `fmt.Printf`, реализует `fmt.Stringer`, то будет использоваться именно он, а не вывод по умолчанию.

Но если мы изменим определение второго поля структуры на `Time` `time.Time`, то увидим то, что ожидали: `{Hello 2009-11-10 00:00:00 +0000 UTC}`.

В Go разрешено также встраивать интерфейсы – см., например, определение `io.ReadWriter`:

```
type ReadWriter interface {
    Reader
    Writer
}
```

Для дополнительного чтения

О флагах в блоге *Gopher Academy*

blog.gopheracademy.com/advent-2019/flags/.

О встраивании на сайте *Effective Go*

golang.org/doc/effective_go.html#embedding.

О методах, интерфейсах и встроенных типах в Go в блоге *Ardan Labs*

ardanlabs.com/blog/2014/05/methods-interfaces-and-embedded-types.html.

Задача 12

Странное число

num.go

```
package main

import (
    "fmt"
)

func main() {
    fmt.Println(0x1p-2)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 0.25

ОБСУЖДЕНИЕ

В Go есть несколько числовых типов. Два основных: целые и с плавающей точкой.

Целые

Это целые числа. В Go имеются типы `int8`, `int16`, `int32`, `int64` и `int`. Имеются также типы без знака: `uint8` и т. д.

С плавающей точкой

Это вещественные числа. В Go имеются типы `float32` и `float64`.

Кроме них, имеются и другие типы, например `complex` и различные числа, определенные в пакете `math/big`.

Встретив числовой литерал, например `3.14`, компилятор Go должен разобрать его и установить конкретный тип (в данном случае `float64`). В спецификации Go¹ определено, как можно записывать числа. Вот несколько примеров.

`num_lit.go`

```
package main

import (
    "fmt"
)

func main() {
    // Целые
    printNum(10) // 10 muna int
    printNum(010) // 8 muna int
    printNum(0x10) // 16 muna int
    printNum(0b10) // 2 muna int
    printNum(1_000) // 1000 muna int <1>

    // С плавающей точкой
    printNum(3.14) // 3.14 muna float64
```

¹ https://golang.org/ref/spec#Lexical_elements.

```

printNum(.2) // 0.2 muna float64
printNum(1e3) // 1000 muna float64
printNum(0x1p-2) // 0.25 muna float64

// Комплексные
printNum(1i) // (0+1i) muna complex128
printNum(3 + 7i) // (3+7i) muna complex128
printNum(1 + 0i) // (1+0i) muna complex128
}

func printNum(n interface{}) {
    fmt.Printf("%v of type %T\n", n, n)
}

```

Знак `_` играет роль разделителя тысяч. Он позволяет записывать большие числа в понятном человеку виде.

Форма `1e3` называется *научной нотацией*.

Форма `0x1p-2` в спецификации Go называется *шестнадцатеричным литералом с плавающей точкой* и согласуется со стандартом IEEE 754 2008. Для вычисления значения нужно выполнить следующие действия:

- вычислить значение до `p` как шестнадцатеричное число. В данном случае `0x1 = 1`;
- вычислить значение после `p` как *2 в этой степени*. В данном случае `2-2 = 0.25`;
- перемножить оба числа. В данном случае `1 * 0.25 = 0.25`.

Для дополнительного чтения

Лексические элементы в спецификации Go
golang.org/ref/spec#Lexical_elements.

«Википедия» о научной нотации
en.wikipedia.org/wiki/Scientific_notation.

«Википедия» о стандарте IEEE 754
en.wikipedia.org/wiki/IEEE_754.

Целые литералы
golang.org/ref/spec#Integer_literals.

Литералы с плавающей точкой
golang.org/ref/spec#Floating-point_literals.

Мнимые литералы
golang.org/ref/spec#Imaginary_literals.

Задача 13

Целые числа и диапазоны

range.go

```
package main

import (
    "fmt"
)

func fibs(n int) chan int {
    ch := make(chan int)

    go func() {
        a, b := 1, 1
        for i := 0; i < n; i++ {
            ch <- a
            a, b = b, a+b
        }
    }()

    return ch
}

func main() {
    for i := range fibs(5) {
        fmt.Printf("%d ", i)
    }
    fmt.Println()
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

В этой программе произойдет взаимоблокировка.

ОБСУЖДЕНИЕ

Чтобы получить значение из канала, можно использовать либо оператор приема ($\leftarrow$ ch), либо цикл `for` по диапазону `range`, в котором будут потребляться все данные из канала.

Как диапазон узнает, что в канале больше нет значений? Ждет закрытия канала. Проблема в показанном выше коде состоит в том, что го-подпрограмма не закрывает канал.

range_close.go

```
package main

import (
    "fmt"
)

func fibs(n int) chan int {
    ch := make(chan int)

    go func() {
        defer close(ch) // <1>
        a, b := 1, 1
        for i := 0; i < n; i++ {
            ch <- a
            a, b = b, a+b
        }
    }()

    return ch
}

func main() {
    for i := range fibs(5) {
        fmt.Printf("%d ", i)
    }
    fmt.Println()
}
```

В случае итерирования по каналу диапазон ведет себя так:

range_rcv.go

```
package main

import (
    "fmt"
)

func fibs(n int) chan int {
    ch := make(chan int)

    go func() {
        defer close(ch)
        a, b := 1, 1
        for i := 0; i < n; i++ {
            ch <- a
            a, b = b, a+b
        }
    }()

    return ch
}

func main() {
    ch := fibs(5)
    for {
        i, ok := <-ch
        if !ok {
            break
        }
        fmt.Printf("%d ", i)
    }
    fmt.Println()
}
```

При использовании этого метода для создания итераторов помните об утечке в go-подпрограммах. Если создать канал `ch := fibs(3)`, а затем потребить из него только одно или два значения, то go-подпрограмма внутри `fibs` останется заблокированной в операции отправки. Поскольку существует ссылка на канал, сборщик мусора не сможет убрать его. Как пишет Дэйв Чини, «никогда не запускайте go-подпрограмму, если не знаете, как она завершится».

Стандартная практика – передавать канал `done` или контекст `context.Context`. При этом код немного усложнится, но вы сохраните контроль над остановкой go-подпрограмм и избежите утечек в go-подпрограммах.

range_ctx.go

```
package main

import (
    "context"
    "fmt"
)

func fibs(ctx context.Context, n int) chan int {
    ch := make(chan int)
    go func() {
        defer close(ch)
        a, b := 1, 1
        for i := 0; i < n; i++ {
            select {
            case ch <- a:
                a, b = b, a+b
            case <-ctx.Done():
                fmt.Println("cancelled")
            }
        }
    }()

    return ch
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    ch := fibs(ctx, 5)
    for i := 0; i < 3; i++ {
        val := <-ch
        fmt.Printf("%d ", val)
    }
    fmt.Println()
    cancel()
}
```

Для дополнительного чтения

Паттерны конкурентности в Go: контекст

blog.golang.org/context.

Паттерны конкурентности в Go: конвейеры и отмена

blog.golang.org/pipelines.

Пакет Context

golang.org/pkg/context/.

Глава «Concurrency» в книге Dave Cheney «Practical Go»

dave.cheney.net/practical-go/presentations/gophercon-israel.html#concurrency.

Задача 14

Несколько личин

multi_assign.go

```
package main

import (
    "fmt"
)

func main() {
    a, b := 1, 2
    b, c := 3, 4
    fmt.Println(a, b, c)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 1 3 4

Обсуждение

Краткое объявление переменных в Go (`:=`) можно использовать в *многозначном* контексте, когда в левой части находится более одной переменной.

Если все переменные в левой части новые, то все в порядке:

```
a, b := 1, 2
```

Если новых переменных нет вовсе, то произойдет ошибка компиляции:

```
a, b := 1, 2
```

```
a, b := 2, 3 // ошибка: нет новых переменных слева от :=
```

А если некоторые переменные уже есть, а другие новые, то все будет в порядке при условии, что типы совпадают. В данном случае в присваивании

```
b, c := 3, 4
```

переменная `b` существует и тип литерала `3` (`int`) совпадает с ее текущим типом. Ну а поскольку `c` – новая переменная, то ошибки нет.

Для дополнительного чтения

Краткое объявление переменных на сайте *Go Tour*
tour.golang.org/basics/10.

Объявления константы в спецификации Go
golang.org/ref/spec#Constant_declarations.

Задача 15

Повесть о двух городах

two_cities.go

```
package main

import (
    "fmt"
)

func main() {
    city1, city2 := "Kraków", "Kraków"
    fmt.Println(city1 == city2)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Не пытайтесь копировать код из PDF-файла в редактор, может случиться, что он не заработает. Для выполнения берите файлы из репозитория.

Программа печатает следующий результат: `false`

ОБСУЖДЕНИЕ

Не пугайтесь, со зрением у вас все в порядке. Эти две строки *выглядят* одинаково. Но если вы напечатаете длину каждой переменной, то окажется, что длина `city1` равна 7, а длина `city2` равна 8.

В задаче 3 «Однажды в Кракове» мы говорили, что в Go строки – это срезы байтов в кодировке UTF-8. У этой кодировки много интересных особенностей, одна из них заключается в том, что некоторые *литеры* (руны) предназначены не для отображения, а для управления, например задания направления текста в двунаправленном (*bidi*) тексте.

В переменной `city1` в позиции 4 находится 1-байтовая руна (ó), а в `city2` там находится руна o и вслед за ней управляющий символ, означающий «добавить акут над предыдущей литерой». Такие способы кодирования называются NFC и NFD.

Строки сравниваются побайтово, потому-то и выводится `false`. Чтобы исправить ошибку, понадобится внешняя библиотека golang.org/x/text/unicode/norm.

two_cities_nfc.go

```
package main

import (
    "fmt"
    "golang.org/x/text/unicode/norm"
)

func main() {
    city1, city2 := "Kraków", "Kraków"
    city1, city2 = norm.NFC.String(city1), norm.NFC.String(city2)
    fmt.Println(city1 == city2)
}
```

Следите за тем, чтобы весь текст, с которым вы работаете, был преобразован в одну и ту же форму нормализации (например, NFC).

Пакеты в каталоге golang.org/x не включены в стандартную библиотеку, но сопровождаются разработчиками ядра Go (и их друзьями).

На golang.org/x много полезных библиотек. Я рекомендую потратить некоторое время, чтобы познакомиться с ними.

Например, в пакете golang.org/x/text/transform есть функция `NewReader`, которая возвращает объект `io.Reader`, преобразующий все, что он читает, в нормальную форму.

Для дополнительного чтения

«Википедия» о двунаправленном тексте

en.wikipedia.org/wiki/Bidirectional_text.

«Википедия» об эквивалентности в Юникоде

en.wikipedia.org/wiki/Unicode_equivalence.

Строки, байты, руны и литеры в Go

blog.golang.org/strings.

«Википедия» о кодировке UTF-8

en.wikipedia.org/wiki/UTF-8.

Задача 16

Что у нас в канале?

chan.go

```
package main

import (
    "fmt"
)

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    <-ch
    close(ch)
    a := <-ch
    b := <-ch
    fmt.Println(a, b)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 2 0

Обсуждение

`ch` – буферизованный канал емкости 2 (это легко проверить, вызвав `cap(ch)`). Мы отправляем в канал `ch` два значения: 1 и 2. Попытавшись отправить еще одно, получим взаимоблокировку.

Затем мы принимаем значение из канала `ch`, закрываем его и принимаем еще два значения. Вопрос: что происходит, когда мы принимаем что-то из закрытого канала?

Ответ: если значения находятся в буфере, то оттуда мы их и получаем; в противном случае мы получим нулевое значение передаваемого по каналу типа. Поэтому `a` получает значение 2, которое находилось в буфере, а `b` – значение 0, т. е. нулевое значение типа `int`.

Как узнать, что мы получили: значение, которое действительно находилось в канале, или нулевое значение из-за того, что канал закрылся? С помощью стандартной конструкции `запятая, ok`.

chan_empty.go

```
package main

import (
    "fmt"
)

func main() {
    ch := make(chan int, 1)
    ch <- 1
    a, ok := <-ch
    fmt.Println(a, ok) // 1 true
    close(ch)
    b, ok := <-ch
    fmt.Println(b, ok) // 0 false
}
```

Для дополнительного чтения

Буферизованные каналы на сайте Go Tour
tour.golang.org/concurrency/3.

Каналы на сайте Effective Go
golang.org/doc/effective_go.html#channels.

Зачем в Go существуют nil-каналы?
medium.com/justforfunc/why-are-there-nil-channels-in-go-9877cc0b2308.

Задача 17

****Int**** ересная строка

strint.go

```
package main

import (
    "fmt"
)

func main() {
    i := 169
    s := string(i)
    fmt.Println(s)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: ☹

ОБСУЖДЕНИЕ

Тип `string` поддерживает преобразование из типа `int`. Целое число при этом рассматривается как руна. Руне `169` соответствует знак копирайта (©). Те, кто допускает эту ошибку, обычно имеют опыт работы на других языках, например Python, где `str(169) → "169"`.

Если вы хотите преобразовать число в строку (или строку в число), то пользуйтесь пакетом `strconv`.

`strint_conv.go`

```
package main

import (
    "fmt"
    "strconv"
)

func main() {
    i := 169
    s := strconv.Itoa(i)
    fmt.Println(s)
}
```

Строки также поддерживают преобразование из байтового среза.

`strint_byte.go`

```
package main

import (
    "fmt"
)

func main() {
    data := []byte{'1', '2', '9'}
    s := string(data)
    fmt.Println(s) // 129
}
```

При преобразовании байтового среза в строку Go копирует срез, попутно выделяя память. В отображениях, где нельзя использовать `[]byte` в качестве ключа, компилятор производит оптимизацию.

strint_map.go

```
package main

import (
    "fmt"
)

func main() {
    m := map[string]int{
        "hello": 3,
    }
    key := []byte{'h', 'e', 'l', 'l', 'o'}
    val := m[string(key)] // выделение памяти не производится
    fmt.Println(val) // 3
}
```

Для дополнительного чтения

Документация по пакету strconv

golang.org/pkg/strconv/.

Пакет runes на golang.org/x

godoc.org/golang.org/x/text/unicode/runenames.

Глава «Using []byte as a Map Key» из книги Dave Cheney «High-Performance Go Workshop»

<https://dave.cheney.net/high-performance-go-workshop/gophercon-2019.html>.

Задача 18

Отложенная работа

job.go

```
package main

import (
    "fmt"
)

type Job struct {
    State string
    done chan struct{}
}

func (j *Job) Wait() {
    <-j.done
}

func (j *Job) Done() {
    j.State = "done"
    close(j.done)
}

func main() {
    ch := make(chan Job)
    go func() {
        j := <-ch
        j.Done()
    }()

    job := Job{"ready", make(chan struct{})} // строка 28
    ch <- job
    job.Wait()
    fmt.Println(job.State)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `ready`

ОБСУЖДЕНИЕ

На первый взгляд, с этим кодом все хорошо. В методах структуры `Job` используется приемник указателя. Тот факт, что вызов `Wait` завершился, говорит нам, что канал был закрыт.

Проблема таится в определении `ch`. Это канал объектов `Job`, а не `*Job`, а это значит, что при отправке переменной `job` по каналу мы на самом деле отправляем ее копию. Канал в Go является *указателеподобным* типом, т. е., хотя внутри го-подпрограммы используется копия `job`, `j.done` указывает на тот же канал, что и `job.done`.

Строки в Go не являются указателеподобными. При вызове `j.Done()` внутри го-подпрограммы мы изменяем значение поля `State` копии `job` в го-подпрограмме. Это изменение не отражается на переменной `job`, объявленной в строке 28.

Решение состоит в том, чтобы сделать канал `ch` типа `*Job`.

`job_ptr.go`

```
package main

import (
    "fmt"
)

type Job struct {
    State string
    done chan struct{}
}

func (j *Job) Wait() {
    <- j.done
}

func (j *Job) Done() {
    j.State = "done"
    close(j.done)
}
```

```
func main() {  
    ch := make(chan *Job)  
    go func() {  
        j := <-ch  
        j.Done()  
    }()  
  
    job := Job{"ready", make(chan struct{})}  
    ch <- &job  
    job.Wait()  
    fmt.Println(job.State)  
}
```

Для дополнительного чтения

В Go нет передачи по ссылке

dave.cheney.net/2017/04/29/there-is-no-pass-by-reference-in-go.

Спецификация типов каналов

golang.org/ref/spec#Channel_types.

Паттерны конкурентности в Go: конвейеры и отмена

blog.golang.org/pipelines.

Каналы на сайте Go Tour

tour.golang.org/concurrency/2.

Задача 19

Ошибка или не ошибка

error.go

```
package main

import (
    "fmt"
)

type OSError int

func (e *OSError) Error() string {
    return fmt.Sprintf("error #%d", *e)
}

func FileExists(path string) (bool, error) {
    var err *OSError // строка 14
    return false, err // TODO
}

func main() {
    if _, err := FileExists("/no/such/file"); err != nil { // строка 19
        fmt.Printf("error: %s\n", err)
    } else {
        fmt.Println("OK")
    }
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `error: <nil>`

ОБСУЖДЕНИЕ

Предложение `if` в строке 19 говорит, что `err != nil`, но при этом `err` печатается как `<nil>`. Кроме того, переменная `err` в строке 14 не инициализирована, т. е. имеет нулевое значение типа указателя, а это и есть `nil`.

Заглянув в файл `src/runtime/runtime2.go` в репозитории исходного кода Go, мы увидим такое определение:

```
type iface struct {  
    tab *itab  
    data unsafe.Pointer  
}
```

- `itab` описывает интерфейс.
- `data` – указатель на значение, реализующее интерфейс.

Интерфейс считается равным `nil`, только если и `itab`, и `data` равны `nil`, но в данном случае это не так.

Решение состоит в том, чтобы *всегда* использовать переменные типа `error` для возврата ошибок. В данном случае строку 14 нужно изменить на `var err error`.

Для дополнительного чтения

Интерфейсы в книге go-internals

github.com/teh-cmc/go-internals/blob/master/chapter2_interfaces/README.md.

Структуры данных в Go: интерфейсы

research.swtch.com/interfaces.

Задача 20

Что в строке?

runes.go

```
package main

import (
    "fmt"
)

func main() {
    msg := "п = 3.14159265358..."
    fmt.Printf("%T ", msg[0])
    for _, c := range msg {
        fmt.Printf("%T\n", c)
        break
    }
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: `uint8 int32`

ОБСУЖДЕНИЕ

Строки в Go представлены в кодировке UTF-8. При доступе к строке с помощью оператора `[]` или метода `len` Go обращается к внутреннему массиву типа `[]byte`. Срез в Go – псевдоним типа `uint8`.

При обходе строки в Go мы получаем литеры Юникода, называемые *рунами*, это псевдоним типа `int32`, поскольку в UTF-8 литеры могут кодироваться числом байтов, не большим 4.

Напечатав руну по спецификатору `%v`, вы увидите числовое значение. Чтобы отобразить символ, используйте спецификатор `%c`. Если вместо руны вы видите `?` или еще какие-то литеры, значит в текущем шрифте этой руны нет. Не существует одного шрифта, в котором были бы представлены все литеры из спецификации Юникода.

Есть несколько способов ввести рунный литерал.

`runes_lit.go`

```
package main
```

```
import (
    "fmt"
)
```

```
func main() {
    r1 := '™'
    fmt.Println(r1) // 8482
    fmt.Printf("%c\n", r1) // ™

    r2 := '\x61' // |x - 2 цифры
    fmt.Printf("%c\n", r2) // a

    r3 := '\u2122' // |u - 4 цифры (8482 в шестнадцатеричном виде)
    fmt.Printf("%c\n", r3) // ™

    r4 := '\U00002122' // |U - 8 цифр
    fmt.Printf("%c\n", r4) // ™
}
```

Для дополнительного чтения

Строки, байты, руны и литеры в Go
blog.golang.org/strings.

Вы и Юникод
betterexplained.com/articles/unicode/.

«Википедия» о Юникоде
en.wikipedia.org/wiki/Unicode.

«Википедия» о кодировке UTF-8
en.wikipedia.org/wiki/UTF-8.

Рунные литералы в спецификации Go
golang.org/ref/spec#Rune_literals.

Таблица литер Юникода
unicode-table.com/.

Задача 21

Двойняшки

init.go

```
package main

import (
    "fmt"
)

func init() {
    fmt.Printf("A ")
}

func init() {
    fmt.Print("B ")
}

func main() {
    fmt.Println()
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: [A](#) [B](#)

ОБСУЖДЕНИЕ

При нормальных обстоятельствах компилятор Go не позволяет определить две функции с одинаковым именем в одном пакете. Но `init` – случай особый. Вот что говорит о нем документация:

«В одном пакете и даже в одном исходном файле может быть определено несколько таких функций. В блоке пакета идентификатор `init` можно использовать только для объявления функций `init`, но сам идентификатор не объявляется. Таким образом, на функции `init` нельзя сослаться ни из какого места программы».

`init` – последняя стадия инициализации пакета, выполняемая после того, как инициализированы импортированные модули и переменные уровня пакета.

Поскольку инициализировать переменные уровня пакета можно с помощью вызовов функций, `init` остается для специальных случаев. Кроме того, старайтесь по возможности избегать переменных уровня пакета. Я редко пишу функции `init` и предпочитаю инициализацию в `main`, где у меня больше контроля над порядком инициализации, обработкой ошибок и протоколированием.

Одна из проблем переменных уровня пакета или `init` – невозможность вернуть код ошибки. Если в `init` произошла ошибка, то лучшее, что можно сделать, – панически завершить программу, но не продолжать работу в ошибочном состоянии. Для этого в некоторых пакетах Go имеется версия `Must` функций `New`. Например, в пакете `regexp` имеется функция `MustCompile`:

«`MustCompile` похожа на `Compile`, но паникует, если выражение невозможно разобрать. Она упрощает безопасную инициализацию глобальных переменных, содержащих откомпилированные регулярные выражения».

Подумайте, следует ли предоставлять функцию `Must` для своих собственных типов.

Для дополнительного чтения

Об использовании функции `init` на сайте *Effective Go*
golang.org/doc/effective_go.html#init.

Инициализация пакета в спецификации Go
golang.org/ref/spec#Package_initialization.

Dave Cheney «*Removing Package Scoped Variables, in Practice*»
dave.cheney.net/2017/06/11/go-without-package-scoped-variables.

Ran Tavory «*Writing Deployable Code (Part 2)*»
medium.com/@rantav/writing-deployable-code-part-two-217bc884c917.

Задача 22

Отсчитайте мне миллион

count.go

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var count int
    var wg sync.WaitGroup
    for i := 0; i < 1_000_000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            count++ // строка 16
        }()
    }
    wg.Wait()
    fmt.Println(count)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: [891239](#)

Другой результат



На вашем компьютере результат может быть иным, но всегда меньше 1 000 000.

Обсуждение

Здесь мы имеем состояние гонки. Операция инкремента целого числа не атомарна, т. е. код может быть прерван посреди операции. Кроме того, существует барьер памяти. Для ускорения работы современные компьютеры сначала пробуют выбрать значение из процессорного кеша и, только если его там нет, начинают долгий путь к основной памяти.

Посмотреть на ассемблерный код [count.go](#) можно, выполнив команду `compile -S count.go`. При этом вы увидите, в частности, код, сгенерированный для предложения `count++` в строке 16:

```
0x0045 00069 (count.go:16) PCDATA $0, $1
0x0045 00069 (count.go:16) PCDATA $1, $4
0x0045 00069 (count.go:16) MOVQ "".&count+56(SP), AX
0x004a 00074 (count.go:16) PCDATA $0, $0
0x004a 00074 (count.go:16) INCQ (AX)
```

Команда `MOVQ` считывает текущее значение счетчика в регистр `AX`. Оно может находиться как в памяти, так и в кеше.

Инструментарий Go умеет обнаруживать такие состояния гонки. Команды `go run` и `go test` поддерживают предназначенный для этой цели флаг `-race`. Попробуем:

```
$ go run -race count.go
=====
WARNING: DATA RACE
Read at 0x00c00011a010 by goroutine 8:
    main.main.func1()
        ./code/count.go:16 +0x6c
```

Previous write at 0x00c00011a010 by goroutine 7:

```
main.main.func1()
./code/count.go:16 +0x82
```

Goroutine 8 (running) created at:

```
main.main()
./code/count.go:14 +0xe4
```

Goroutine 7 (finished) created at:

```
main.main()
./code/count.go:14 +0xe4
```

Чтобы решить проблему, можно воспользоваться типом `sync.Mutex`, который гарантирует, что в каждый момент времени обновлять `count` может только одна go-подпрограмма.

count_mu.go

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var count int
    var wg sync.WaitGroup
    var m sync.Mutex

    for i := 0; i < 1_000_000; i++ {
        wg.Add(1)
        go func() {
            m.Lock()
            defer m.Unlock()
            defer wg.Done()
            count++
        }()
    }
    wg.Wait()
    fmt.Println(count)
}
```

Другой способ – воспользоваться пакетом `sync/atomic`, который предоставляет атомарные операции: они работают быстрее мьютекса, но использовать их труднее.

`count_atomic.go`

```
package main
```

```
import (  
    "fmt"  
    "sync"  
    "sync/atomic"  
)
```

```
func main() {  
    var count int64 // Пакет atomic работаем с int64, а не int  
    var wg sync.WaitGroup  
  
    for i := 0; i < 1_000_000; i++ {  
        wg.Add(1)  
        go func() {  
            defer wg.Done()  
            atomic.AddInt64(&count, 1)  
        }()  
    }  
    wg.Wait()  
    fmt.Println(count)  
}
```

Хотя Go и пытается абстрагировать оборудование, все равно вы должны изучать его и понимать, как оно работает.

Для дополнительного чтения

«Википедия» о состоянии гонки
en.wikipedia.org/wiki/Race_condition.

Атомарные и неатомарные операции
preshing.com/20130618/atomic-vs-non-atomic-operations/.

Введение в детектор гонок в Go
blog.golang.org/race-detector.

Барьеры памяти

mechanical-sympathy.blogspot.com/2011/07/memory-barriersfences.html.

«Википедия» о барьерах памяти

en.wikipedia.org/wiki/Memory_barrier.

Планирование в Go: часть I – планировщик ОС

ardanlabs.com/blog/2018/08/scheduling-in-go-part1.html.

Компьютерные задержки в понятном человеку масштабе

twitter.com/srigi/status/917998817051541504.

Задача 23

Кто следующий?

next_id.go

```
package main

import (
    "fmt"
)

var (
    id = nextID()
)

func nextID() int {
    id++
    return id
}

func main() {
    fmt.Println(id)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Этот код не компилируется.

Обсуждение

При попытке собрать программу возникает ошибка:

```
$ go build next_id.go
# command-line-arguments
./next_id.go:8:2: initialization loop:
    /home/miki/Projects/go-brain-teasers/code/next_id.go:8:2: id refers to
    /home/miki/Projects/go-brain-teasers/code/
    next_id.go:11:6: nextID refers to
    /home/miki/Projects/go-brain-teasers/code/next_id.go:8:2: id
```

Инициализация за циклируется. Значение `id` зависит от `nextID`, при вычислении которого используется значение `id`, которое...

Компилятор Go пытается обнаруживать такие циклы, но бывает, что не может этого сделать (это называется *скрытыми зависимостями*).

Следите за тем, чтобы инициализация пакета была простой и понятной. Старайтесь избегать переменных уровня пакета и по возможности откладывайте инициализацию до `main`.

Для дополнительного чтения

Инициализация пакета в спецификации Go
golang.org/ref/spec#Package_initialization.

Dave Cheney «*Removing Package Scoped Variables, in Practice*»
dave.cheney.net/2017/06/11/go-without-package-scoped-variables.

Задача 24

Ох уж эти флаги

flag.go

```
package main

import (
    "fmt"
)

type Flag int

func main() {
    var i interface{} = 3
    f := i.(Flag)
    fmt.Println(f)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Этот код паникует.

Обсуждение

Хотя `Flag` – по существу, тот же `int`, компилятор Go рассматривает его как отдельный тип, и утверждение о типе `(i.(Flag))` паникует. Это еще один случай, где может быть полезна конструкция *запятая, ok*.

flag_ok.go

```
package main

import (
    "fmt"
)

type Flag int // строка 7

func main() {
    var i interface{} = 3
    f, ok := i.(Flag)
    if !ok {
        fmt.Println("not a Flag")
        return
    }
    fmt.Println(f)
}
```

Строку 7 можно заменить на `Flag = int`. В таком случае `Flag` – псевдоним типа, и код будет работать. Но теперь мы можем использовать `int` всюду, где нужен `Flag`, и система типов нас не защитит.

Для дополнительного чтения

Псевдонимы типов

golang.org/doc/go1.9#language.

«Codebase Refactoring (with Help from Go)» (объясняется, зачем нужны псевдонимы типов)

talks.golang.org/2016/refactor.article.

Объявления типов в спецификации Go

golang.org/ref/spec#Type_declarations.

Утверждения о типах в спецификации Go

golang.org/ref/spec#Type_assertions.

Переключатель типа на сайте Effective Go

golang.org/doc/effective_go.html#type_switch.

Преобразования интерфейсов и утверждения о типах на сайте Effective Go

golang.org/doc/effective_go.html#interface_conversions.

Задача 25

Даю разрешение

iota.go

```
package main

import (
    "fmt"
)

const (
    Read = 1 << iota
    Write
    Execute
)

func main() {
    fmt.Println(Execute)
}
```

Угадайте результат



Попробуйте угадать результат, не переворачивая страницу.

Программа печатает следующий результат: 4

ОБСУЖДЕНИЕ

`iota` – реализация перечислимого типа в Go. Его можно использовать в объявлении `const`. `iota` увеличивается на 1 для каждой константы в пределах одной группы. Если вы укажете любую операцию над `iota` (например, сложение), то после выполнения `iota` увеличится. `<<` – оператор поразрядного сдвига влево.

- 1 в двоичной 8-разрядной форме записывается как `00000001`.
- `1<<3` равно `00001000`, или 8.

В данном случае мы начинаем с `Read = 1<<iota`, и это транслируется в `1<<0 → 0001`, т. е. 1. Затем производится неявное присваивание `1<<1 → 0010` константе `Write`, которая становится равной 2, и наконец, неявное присваивание `1<<2 → 0100` константе `Execute`, которая становится равной 4.

Чаще всего эти поразрядные операции используются для флагов. Одним байтом можно представить восемь флагов, соединив их оператором `|` (поразрядное ИЛИ) (это называется *битовой маской*). Чтобы проверить, поднят ли флаг, нужно использовать оператор `&` (поразрядное И).

`iota_check.go`

```
package main

import (
    "fmt"
)

const (
    Read = 1 << iota
    Write
    Execute
)

func main() {
    mask := Read | Execute
    if mask&Execute == 0 {
        fmt.Println("can't execute")
    }
}
```

```

    } else {
        fmt.Println("can execute") // будет напечатано
    }
    if mask&Write == 0 {
        fmt.Println("can't write") // будет напечатано
    } else {
        fmt.Println("can write")
    }
}

```

Значениями типа `iota` являются числа. Иногда нужно, чтобы у констант было понятное человеку представление. Тогда константам назначается тип, и для этого типа реализуется интерфейс `fmt.Stringer`.

`iota_str.go`

```

package main

import (
    "fmt"
)

type FilePerm uint16 // 16 флагов достаточно

const (
    Read FilePerm = 1 << iota
    Write
    Execute
)

// Тут String реализует интерфейс fmt.Stringer
func (p FilePerm) String() string {
    switch p {
    case Read:
        return "read"
    case Write:
        return "write"
    case Execute:
        return "execute"
    }
    return fmt.Sprintf("unknown FilePerm: %d", p)
    // не используйте здесь %s :)
}

```

```
func main() {  
    fmt.Println(Execute) // выполнить  
    fmt.Printf("%d\n", Execute) // 4  
}
```

Оставляю вам в качестве упражнения реализацию метода `String()` `string`, который поддерживает битовые маски. 😊

Для дополнительного чтения

«Википедия» о перечислимом `type`
en.wikipedia.org/wiki/Enumerated_type.

`iota`
github.com/golang/go/wiki/Iota.

Константы на сайте *Effective Go*
golang.org/doc/effective_go.html#constants.

«Википедия» о поразрядных операциях
en.wikipedia.org/wiki/Bitwise_operation.

Предметный указатель

Symbols

%c 84
-race флаг 92
%v спецификатор 50

А

атомарные операции 94

Б

байты
 кодировка ASCII 24
 кодировка UTF-8 24, 66
 преобразование строк 74
 сравнение строк 66
битовая маска 104
буферизованные каналы 70
время
 JSON-сериализация 42
 и вывод типа 32
 монотонные часы 43
 ресурсы 43
 сравнение 43

В

встраивание 50
вывод типа 32

Д

двунаправленный текст 66
== для сравнения времени 43

З

зависимости скрытые 98
замыкание 38
запятая, ok 22, 70, 100

И

идентификаторы, Юникод 18
инициализация
 в main 88
 зацикливание 98
 переменных уровня пакета 88
интерфейсы
 nil 82
 ресурсы 51, 82, 101
исходные строки 30

К

“” (кавычки) для
 интерпретированных строк 30
каналы
 nil 28
 буферизованные 70
 и диапазоны значений 58
 и спецификации типов 78
конвейеры, ресурсы 61, 79
конкурентность, ресурсы 61, 79
константы
 и iota 104
 понятное человеку
 представление 105
 ресурсы 19, 64, 106

Л

лямбда-функции см. замыкания

М

методы

встраивание 50

ресурсы 51

мнимые литералы 55

многострочные строки 30

мьютексы 93

Н

научная нотация 55

нормализация 66

О

область видимости переменных 39

`` (обратные апострофы) для исходных строк 30

<< оператор поразрядного сдвига влево в `iota` 104

отображения

`nil` 22, 28

ресурсы 22

ошибки

`init` и переменные уровня пакета 88

П

память

барьеры 95

состояния гонки 92

передача по ссылке 79

перечислимый тип, `iota` 104

`&` (поразрядное И) 104

`|` (поразрядное ИЛИ) 104

поразрядные операции 104

Р

руны

и кодировка UTF-8 24, 66, 84

преобразование 74

ресурсы 25

С

символы

отображение 84

состояния гонки 92

срезы

и `nil` 22

преобразование в строку 75

ресурсы 47

строки

интерпретированные 30

исходные 30

кодировка UTF-8 24

многострочные 30

побайтовое сравнение 66

преобразование 74

ресурсы 25

структуры

`Log` 50

печать полей 50

Т

тайм-аут 32

типы

встраивание 50

вывод 32

переключатель 101

преобразование 74

ресурсы 79, 101, 106

Ф

`{}` (фигурные скобки) для открытия области видимости переменных 39

флаги

поразрядные операции 104

ресурсы 51

Ц

целые

без знака 54

операции между целым и числом
с плавающей точкой 18
преобразование 74
ресурсы 55
типы 54

Ч

часы
системные и монотонные 43
числа с плавающей точкой
математические операции над
константами 18
ресурсы 35, 55
типы JSON 42
точность 34
шестнадцатеричные
литералы 55

Ю

Юникод
идентификаторы 18
и интерпретированные
строки 30
нормализация 66
отображение символов 84
ресурсы 25
таблица литер 85
эквивалентность 67

А

append 46
ASCII
байты 24

С

complex тип данных 54
context 60

Д

data 82
done канал 60

F

float32 54
float64 54

G

golang.org/x пакеты 66
go-подпрограммы
замыкания 38
и каналы 58
и состояния гонки 92
утечки 59

I

InDelta 34
init
ресурсы 89
int 54, 100
int8 54
int16 54
int32 54, 84
int64 54
IsNaN 34
ISO 8601 43
itab 82

J

Job и спецификация типа канала 78
JSON
ресурсы 43
сериализация времени 42

L

len
дописывание срезов 46
и отображения nil 22
и руны 84
Log структура 50

M

main
инициализация в 88, 98

Marshaler интерфейс 42
math/big пакет 34, 54
MustCompile 88

N

NaN 34
NewReader 67
NFD форма нормализации 66
nil
 безопасные типы 22
 интерфейсы 82
 каналы 28
 отображения 22, 28
 ресурсы 28, 71

R

runenames пакет 75

S

shopspring/decimal пакет 34

stretchr/testify пакет 34
Stringer интерфейс 105
sync/atomic пакет 94
sync.Mutex пакет 93

T

time.Duration 32
time.Time 42

U

unicode/utf8 пакет 24
Unmarshaler интерфейс 42
UTF-8 кодировка
 длина строки 24
 нормализация 66
 ресурсы 25
 руны 24, 66, 84
 сравнение длин строк 66

Книги издательства «ДМК Пресс» можно купить оптом и в розницу
на складе издательства по адресу:
Москва, ул. Электродная, д. 2, стр. 12, офис 7,
тел. +7 (499) 322–19–38,
а также заказать на сайте www.dmkpress.com
с доставкой в любой регион РФ.

Мики Тебека

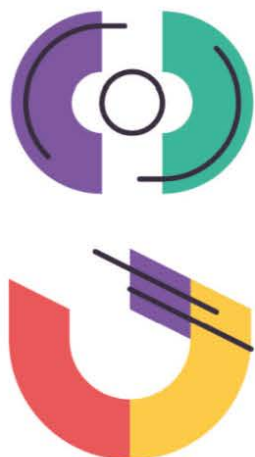
Хорошо ли вы знаете Go?

Главный редактор *Мовчан Д. А.*
Зам. главного редактора *Яценков В. С.*
editor@dmkpress.com
Перевод *Слинкин А. А.*
Корректор *Абросимова Л. А.*
Верстка *Луценко С. В.*
Дизайн обложки *Мовчан А. Г.*

Формат 60×90 1/16.
Гарнитура «PT Serif». Печать цифровая.
Усл. печ. л. 7. Тираж 200 экз.

Веб-сайт издательства: www.dmkpress.com

Программисты любят язык Go (golang) за эффективность и простоту. Но это не значит, что программирование на Go лишено странностей. Например, скрытые зависимости могут заставить компилятор врасплох, а некоторые преобразования типов отличаются от принятых в таких популярных языках, как Python.



Книга содержит 25 коротких нестандартных примеров кода для проверки вашего понимания языка Go. Решение задач с подвохом – не только надежный способ овладеть новыми знаниями, чтобы лучше справляться с работой, но и прекрасное развлечение. Задачи основаны как на собственном опыте автора по отладке программ, так и на опыте других людей.

Ознакомьтесь с коротким кодом, написанным на Go, попробуйте угадать, что он выведет на печать при выполнении, запустите программу, а затем переверните страницу и прочитайте ответ и объяснение.

Проверьте свои навыки работы с целыми числами, строками и символами Юникода, оцените знание поведения компилятора и многих других нюансов программирования.

Предполагается, что вы в какой-то мере знакомы с языком Go и имеете опыт программирования на нем.

Замечание: в книге используется версия Go 1.14.1, в последующих версиях результаты могут быть иными.



ISBN 978-5-93700-362-1



9 785937 003621 >